

Dokumentacja tworzenia srodowiska chmurowego w Google Cloud

Dane dokumentu

- Tytuł: Dokumentacja tworzenia srodowiska chmurowego w Google Cloud dla systemu rezerwacji sal
- Wersja: 1.1
- Data: 2026-01-21
- Projekt: System rezerwacji sal i zasobow (GKE + Cloud SQL + Cloud Spanner)
- Zakres: utworzenie zasobow chmurowych, budowa obrazow i wdrozenie na GKE

Spis tresci

1. Cel i zakres dokumentu
2. Wymagania projektu i kontekst
3. Zalozenia architektoniczne
4. Wymagania wstepne
5. Konwencje nazewnictwa i parametry globalne
6. Utworzenie projektu Google Cloud i powiazanie z billingiem
7. Konfiguracja IAM (role i konta serwisowe)
8. Aktywacja wymaganych API
9. Konfiguracja sieci (VPC i podsieci)
10. Artifact Registry (repozytorium obrazow)
11. Cloud SQL (PostgreSQL)
12. Cloud Spanner
13. Google Kubernetes Engine (GKE)
14. Workload Identity i mapowanie kont
15. Konfiguracja bezpieczenstwa i sekretow
16. Budowanie i publikacja obrazow (Cloud Build)
17. Wdrozenie aplikacji na GKE (Backend i Frontend)
18. Obserwowalnosc (Monitoring i Logging)
19. Walidacja i checklisty
20. Aneks: zestaw polecen z parametrami

1. Cel i zakres dokumentu

Celem dokumentu jest przedstawienie szczegolowej, akademickiej instrukcji krok po kroku dla utworzenia srodowiska chmurowego w Google Cloud, niezbednego do uruchomienia systemu rezerwacji sal. Dokument obejmuje przygotowanie infrastruktury zgodnej z wymaganiami projektu:

- warstwa aplikacji na Google Kubernetes Engine (GKE),
- relacyjna baza danych na Cloud SQL (PostgreSQL),
- globalna baza rezerwacji na Cloud Spanner.

Zakres obejmuje tworzenie zasobow, konfiguracje IAM, sieci i rejestrow obrazow, a takze budowanie obrazow i wdrozenie aplikacji na GKE. W miejscu, gdzie wdrozenie aplikacji jest niezbedne do weryfikacji polaczen, przedstawiono metody testowe.

2. Wymagania projektu i kontekst

Wymagania funkcjonalne wynikaja z zalozen projektu: system webowy rezerwacji sal w firmie oraz integracja danych lokalnych i globalnych. Wymagania techniczne infrastruktury:

- GKE jako platforma uruchomieniowa serwisow.

- Cloud SQL jako relacyjna baza danych (lokalne rezerwacje i uzytkownicy).
- Cloud Spanner jako globalny magazyn rezerwacji (wspoldzielony miedzy oddzialami).

Zalozony przeplyw:

- 1) Uzytkownik loguje sie do aplikacji (serwis w GKE).
- 2) System sprawdza dostepnosc sal w Cloud SQL.
- 3) Rezerwacje globalne zapisywane sa w Cloud Spanner, a administrator ma widok zbiorczy.

3. Zalozenia architektoniczne

Architektura docelowa (opis logiczny):

- Warstwa aplikacji: kontenery uruchomione na GKE, rozdzielone na backend (API) i frontend.
- Cloud SQL: baza PostgreSQL dla danych lokalnych (uzytkownicy, pokoje, rezerwacje w oddziale).
- Cloud Spanner: tabela GlobalReservations dla rezerwacji globalnych.

Schemat (uproszczony, tekstowy):

```
[Uzytkownik] -> [Frontend (GKE)] -> [Backend (GKE)] -> [Cloud SQL]
|--> [Cloud Spanner]
```

4. Wymagania wstepne

4.1 Konto i uprawnienia

- Konto Google z aktywnym billingiem.
- Uprawnienia typu Owner lub Administrator projektu do tworzenia zasobow.

4.2 Narzedzia lokalne

- Google Cloud SDK (gcloud) zainstalowany lokalnie.
- kubectl (wraz z gcloud components install kubectl lub oddzielnie).
- Docker (do budowania obrazow) - wymagany przy pozniejszym wdrozeniu aplikacji.

4.3 Autoryzacja

Zalecane logowanie:

- gcloud auth login
- gcloud auth application-default login (dla narzedzi i bibliotek korzystajacych z ADC)

4.4 Wersje narzedzi i kompatybilnosc

Zalecane minimalne wersje:

- gcloud SDK >= 470.0.0
- kubectl zgodny z wersja klastra (roznica nie wieksza niz 1 wersja minor)
- Docker Desktop >= 4.x

Sprawdzenie wersji:

```
gcloud version
kubectl version --client
docker --version
```

5. Konwencje nazewnictwa i parametry globalne

Zaleca sie stosowanie spojnego nazewnictwa zasobow. Przykladowe konwencje:

- Project ID: room-booking-prod
- Region: europe-central2 (Warszawa)
- GKE cluster: room-booking-gke
- Cloud SQL: room-booking-sql
- Cloud Spanner: room-booking-global
- Artifact Registry: room-booking

Dodatkowo stosuj etykiety zasobow (labels) do rozliczen i audytu, np. env=dev, owner=team3, cost-center=chmura.

Przyklad:

```
gcloud projects update $env:PROJECT_ID --update-labels=env=dev,owner=team3
```

Parametry globalne ustawiane w CLI (przyklad):

```
$env:PROJECT_ID="room-booking-prod"
$env:REGION="europe-central2"
```

W Cloud Shell (bash) stosuj skladnie:

```
export PROJECT_ID="room-booking-prod"
export REGION="europe-central2"
```

6. Utworzenie projektu Google Cloud i powiazanie z billingiem

6.1 Utworzenie projektu (CLI)

```
gcloud projects create $env:PROJECT_ID --name="room-booking"
```

6.2 Powiazanie z kontem rozliczeniowym

```
gcloud billing accounts list
```

Wybrany BILLING_ID przypisz do projektu:

```
gcloud billing projects link $env:PROJECT_ID --billing-account=BILLING_ID
```

6.3 Ustawienie aktywnego projektu

```
gcloud config set project $env:PROJECT_ID
```

6.4 Ustawienie domyslnego regionu i strefy

```
gcloud config set compute/region $env:REGION
gcloud config set compute/zone europe-central2-a
```

7. Konfiguracja IAM (role i konta serwisowe)

7.1 Zasada najmniejszych uprawnień

W srodowisku produkcyjnym zaleca sie rozdzielic uprawnienia:

- Administrator projektu: tworzenie zasobow.
- Developer/Operator: wdrazanie i utrzymanie.
- Service Accounts: dostep aplikacji do baz.

7.2 Utworzenie konta serwisowego dla backendu

```
gcloud iam service-accounts create room-booking-backend --display-name="Room Booking Backend"
```

7.3 Nadanie rol do Cloud SQL i Spanner

```
gcloud projects add-iam-policy-binding $env:PROJECT_ID
--member="serviceAccount:room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com"
--role="roles/cloudsql.client"
```

```
gcloud projects add-iam-policy-binding $env:PROJECT_ID
--member="serviceAccount:room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com"
--role="roles/spanner.databaseUser"
```

7.4 (Opcjonalnie) Rozdzielenie uprawnień

W srodowiskach akademickich/produkcyjnych mozna rozdzielic role:

- Cloud SQL: roles/cloudsql.client
- Spanner: roles/spanner.databaseUser
- Artifact Registry: roles/artifactregistry.reader (dla pull) lub writer (dla push)

7.5 Audyt i historia zmian IAM

Zmiany IAM są logowane w Cloud Audit Logs. Dla dokumentacji akademickiej warto:

- sprawdzić, czy logi administracyjne są włączone,
- określić okres retencji logów,
- dodać alerty na zmiany w politykach IAM.

8. Aktywacja wymaganych API

Włącz wymagane usługi:

```
gcloud services enable container.googleapis.com sqladmin.googleapis.com spanner.googleapis.com
artifactregistry.googleapis.com iam.googleapis.com
```

9. Konfiguracja sieci (VPC i podsieci)

9.1 Podejście rekomendowane

Dla środowiska edukacyjnego można użyć sieci domyślnej, lecz w produkcji zaleca się dedykowaną VPC z kontrolą ruchu.

9.2 Utworzenie VPC (opcjonalnie, produkcyjnie)

```
gcloud compute networks create room-booking-vpc --subnet-mode=custom
```

```
gcloud compute networks subnets create room-booking-subnet --network=room-booking-vpc
--region=$env:REGION --range=10.10.0.0/16
```

9.3 Reguły zapory (firewall)

W praktyce GKE wymaga ruchu wewnętrznego między węzłami i podami. Przykład dla sieci custom:

```
gcloud compute firewall-rules create allow-internal --network=room-booking-vpc
--allow=tcp,udp,icmp --source-ranges=10.10.0.0/16
```

Uwaga: Cloud SQL z prywatnym IP wymaga dodatkowych konfiguracji (Private Service Access). Szczegóły w sekcji 9.4.

9.4 Private Service Access (opcjonalnie, zalecane produkcyjnie)

Kroki dla VPC peering z usługami Google (wymagane dla prywatnego IP Cloud SQL):

```
gcloud services enable servicenetworking.googleapis.com
```

```
gcloud compute addresses create google-managed-services-room-booking --global
--purpose=VPC_PEERING --addresses=10.20.0.0 --prefix-length=16 --network=room-booking-vpc
```

```
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com
--ranges=google-managed-services-room-booking --network=room-booking-vpc
```

Po zestawieniu peeringu instancje Cloud SQL można tworzyć z prywatnym IP:

```
gcloud sql instances create room-booking-sql --database-version=POSTGRES_15 --cpu=1
--memory=3840MiB --region=$env:REGION --network=room-booking-vpc --no-assign-ip
```

10. Artifact Registry (repozytorium obrazów)

Repozytorium przechowuje obrazy Docker backendu i frontendu.

10.1 Utworzenie repozytorium

```
gcloud artifacts repositories create room-booking --repository-format=docker
--location=$env:REGION
```

10.2 Konfiguracja Docker do push/pull

```
gcloud auth configure-docker $env:REGION-docker.pkg.dev
```

10.3 Uprawnienia do repozytorium

Najczęstsze role:

- roles/artifactregistry.reader (pobieranie obrazów)

- roles/artifactregistry.writer (push i pull)
- roles/artifactregistry.admin (zarządzanie repozytorium)

11. Cloud SQL (PostgreSQL)

11.1 Utworzenie instancji

```
gcloud sql instances create room-booking-sql --database-version=POSTGRES_15 --cpu=1
--memory=3840MiB --region=$env:REGION
```

Uwagi akademickie:

- CPU i RAM można skalować wraz z obciążeniem.
- Zalecane jest włączenie automatycznych backupów i PITR (point-in-time recovery).

11.2 Utworzenie bazy i użytkownika

```
gcloud sql databases create room_booking --instance=room-booking-sql
```

```
gcloud sql users create room_user --instance=room-booking-sql --password=STRONG_PASS
```

11.3 Pobranie connection name

```
gcloud sql instances describe room-booking-sql --format="value(connectionName)"
```

Wynik zostanie użyty przez Cloud SQL Auth Proxy w GKE.

11.4 Import schematu

Metoda 1: Cloud SQL Studio w konsoli (rekomendowane edukacyjnie).

Metoda 2: lokalne psql:

```
gcloud sql connect room-booking-sql --user=room_user --database=room_booking
```

W psql:

```
\i C:/Users/mziar/Desktop/SYS_CHM/docker/postgres/init.sql
```

11.5 Uwagi o połączeniu prywatnym

Dla produkcji zaleca się prywatne IP (Private Service Access), co wymaga:

- utworzenia zakresu adresacji dla usługi,
- połączenia VPC z Cloud SQL,
- konfiguracji GKE w tej samej VPC.

11.6 Backupy, PITR i okno serwisowe

Rekomendowane ustawienia:

```
gcloud sql instances patch room-booking-sql --backup-start-time=03:00
--enable-point-in-time-recovery --retained-backups-count=7
```

```
gcloud sql instances patch room-booking-sql --maintenance-window-day=SAT
--maintenance-window-hour=4
```

11.7 Parametry instancji i autoskalowanie storage

Przykładowe ustawienia:

```
gcloud sql instances patch room-booking-sql --storage-auto-increase --storage-size=20
```

12. Cloud Spanner

12.1 Konfiguracja instancji

Spanner wymaga wyboru konfiguracji regionalnej lub multi-regionalnej.

Dla zgodności z regionem GKE: regional-europe-central2.

Sprawdzenie konfiguracji:

```
gcloud spanner instance-configs list
```

Utworzenie instancji:

```
$env:SPANNER_CONFIG="regional-europe-central2"
```

```
gcloud spanner instances create room-booking-global --config=$env:SPANNER_CONFIG --nodes=1  
--description="Room Booking Global"
```

12.2 Utworzenie bazy danych i schematu

Przygotuj plik DDL (spanner.ddl):

```
CREATE TABLE GlobalReservations (  
  ReservationId STRING(36) NOT NULL,  
  RoomId STRING(36) NOT NULL,  
  UserId STRING(36) NOT NULL,  
  StartTime TIMESTAMP NOT NULL,  
  EndTime TIMESTAMP NOT NULL,  
  Status STRING(16) NOT NULL,  
  CreatedAt TIMESTAMP NOT NULL,  
  CanceledAt TIMESTAMP  
) PRIMARY KEY (ReservationId);
```

Następnie utwórz bazę:

```
gcloud spanner databases create room_booking_global --instance=room-booking-global  
--ddl-file=spanner.ddl
```

12.3 Uprawnienia

Aplikacja potrzebuje roli:

- roles/spanner.databaseUser (do wykonywania zapytań i modyfikacji danych)

12.4 Skalowanie i koszty

Spanner skaluje się poprzez liczbę węzłów (nodes) lub processing units. Minimalnie 1 node.

Przykład zmiany rozmiaru:

```
gcloud spanner instances update room-booking-global --nodes=2
```

W dokumentacji akademickiej warto opisać zależność kosztów od liczby węzłów i regionu.

13. Google Kubernetes Engine (GKE)

13.1 Tryb klastra

Dla projektu edukacyjnego zalecany jest tryb Standard (większa kontrola i transparentność kosztów).

13.2 Utworzenie klastra

```
gcloud container clusters create room-booking-gke --region=$env:REGION --num-nodes=2  
--workload-pool=$env:PROJECT_ID.svc.id.goog --release-channel=regular
```

13.3 Pobranie credentials

```
gcloud container clusters get-credentials room-booking-gke --region=$env:REGION
```

13.4 Uwagi o sieciach i IP range

W przypadku niestandardowej VPC należy podać parametry sieci i podsieci w poleceniu tworzenia klastra.

13.5 Autoskalowanie i parametry węzłów

Przykładowa konfiguracja autoskalowania dla domyślnego node pool:

```
gcloud container clusters create room-booking-gke --region=$env:REGION --num-nodes=1  
--enable-autoscaling --min-nodes=1 --max-nodes=3 --machine-type=e2-standard-2  
--disk-type=pd-balanced --disk-size=100 --workload-pool=$env:PROJECT_ID.svc.id.goog  
--release-channel=regular
```

14. Workload Identity i mapowanie kont

Workload Identity pozwala kontenerom korzystać z GCP Service Account bez kluczy JSON.

14.1 Utworzenie Kubernetes Service Account

```
kubectl create serviceaccount backend-sa
```

14.2 Powiazanie KSA z GSA

```
kubectl annotate serviceaccount backend-sa  
iam.gke.io/gcp-service-account=room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com  
  
gcloud iam service-accounts add-iam-policy-binding  
room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com  
--role=roles/iam.workloadIdentityUser  
--member="serviceAccount:$env:PROJECT_ID.svc.id.goog[default/backend-sa]"
```

14.3 Weryfikacja mapowania

```
kubectl describe serviceaccount backend-sa  
gcloud iam service-accounts get-iam-policy  
room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com
```

15. Konfiguracja bezpieczeństwa i sekretów

15.1 Sekrety aplikacji

Dane wrażliwe (hasła DB, JWT) powinny być przechowywane w Secret Manager lub w Kubernetes Secrets.

Przykład w Kubernetes (tylko do celów edukacyjnych):

```
kubectl create secret generic backend-secrets --from-literal=JWT_SECRET=...  
--from-literal=POSTGRES_PASSWORD=...
```

15.2 Zasady bezpieczeństwa

- Minimalne uprawnienia IAM.
- Wylaczenie publicznego dostępu do Cloud SQL (preferowane).
- Wymuszenie TLS dla połączeń zewnętrznych.

15.3 Secret Manager (zalecane)

Alternatywa wobec Kubernetes Secrets:

```
gcloud secrets create jwt-secret --replication-policy=automatic  
gcloud secrets versions add jwt-secret --data-file=-
```

Wdrożenie w GKE wymaga odpowiednich uprawnień IAM i mechanizmu podania sekretu do poda (np. CSI Driver).

16. Budowanie i publikacja obrazów (Cloud Build)

16.1 Włączenie API i uprawnienia

W Cloud Shell zaleca się wykorzystanie Cloud Build do budowy obrazów. Wymagane API:

```
gcloud services enable cloudbuild.googleapis.com
```

Użytkownik uruchamiający build powinien mieć rolę:

```
gcloud projects add-iam-policy-binding $PROJECT_ID --member="user:USER_EMAIL"  
--role="roles/cloudbuild.builds.editor"
```

16.2 Uprawnienia do Artifact Registry dla Cloud Build

Cloud Build musi mieć prawo do push obrazu. Nadaj rolę writer odpowiedniemu service account:

```
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format='value(projectNumber)')
```

```
gcloud projects add-iam-policy-binding "$PROJECT_ID" \  
--member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \  
--role="roles/artifactregistry.writer"
```

```
gcloud projects add-iam-policy-binding "$PROJECT_ID" \  
--member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \  
--role="roles/artifactregistry.writer"
```

Opcjonalnie (logi Cloud Build):

```
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
  --role="roles/logging.logWriter"
```

16.3 Budowa i publikacja backendu

Przygotuj zmienne:

```
export REPO="$REGION-docker.pkg.dev/$PROJECT_ID/room-booking"
export BACKEND_IMAGE="$REPO/backend:1"
```

Konfiguracja Cloud Build:

```
cat > cloudbuild-backend.yaml <<EOF
steps:
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', '${BACKEND_IMAGE}', '-f', 'apps/backend/Dockerfile', '.']
images:
- '${BACKEND_IMAGE}'
EOF
```

Uruchom build:

```
gcloud builds submit --config cloudbuild-backend.yaml .
```

16.4 Budowa i publikacja frontendu

Po wdrożeniu backendu pobierz IP load balancera:

```
BACKEND_LB=$(kubectl get svc backend-service -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

Zbuduj frontend z poprawnym URL API:

```
export FRONTEND_IMAGE="$REPO/frontend:1"
```

```
cat > cloudbuild-frontend.yaml <<EOF
steps:
- name: 'gcr.io/cloud-builders/docker'
  args:
  ['build', '-t', '${FRONTEND_IMAGE}', '-f', 'apps/frontend/Dockerfile', '--build-arg', 'VITE_API_URL=ht
tp://${BACKEND_LB}', '.']
images:
- '${FRONTEND_IMAGE}'
EOF
```

```
gcloud builds submit --config cloudbuild-frontend.yaml .
```

17. Wdrożenie aplikacji na GKE (Backend i Frontend)

17.1 Sekrety i konfiguracja backendu

Sekrety aplikacji (JWT, hasło DB) przechowuj w Kubernetes Secrets:

```
kubectl delete secret backend-secrets --ignore-not-found
kubectl create secret generic backend-secrets \
  --from-literal=JWT_SECRET="$(openssl rand -hex 32)" \
  --from-literal=POSTGRES_PASSWORD="STRONG_PASS"
```

ConfigMap z parametrami backendu (bez SPANNER_EMULATOR_HOST w produkcji):

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
data:
  PORT: "4000"
  NODE_ENV: "production"
  POSTGRES_HOST: "127.0.0.1"
  POSTGRES_PORT: "5432"
  POSTGRES_DB: "room_booking"
  POSTGRES_USER: "room_user"
  SPANNER_PROJECT_ID: "$PROJECT_ID"
  SPANNER_INSTANCE_ID: "room-booking-global"
```



```
SPANNER_DATABASE_ID: "room_booking_global"
```

17.2 Backend Deployment i Cloud SQL Proxy

Zalecany wzorzec to sidecar Cloud SQL Proxy:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: room-booking-backend
  template:
    metadata:
      labels:
        app: room-booking-backend
    spec:
      serviceAccountName: backend-sa
      containers:
        - name: api
          image: $BACKEND_IMAGE
          ports:
            - containerPort: 4000
          envFrom:
            - configMapRef:
                name: backend-config
            - secretRef:
                name: backend-secrets
        - name: cloud-sql-proxy
          image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.11.4
          args:
            - "--address=127.0.0.1"
            - "--port=5432"
            - "$CONNECTION_NAME"
          securityContext:
            runAsNonRoot: true
```

Service typu LoadBalancer:

```
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  type: LoadBalancer
  selector:
    app: room-booking-backend
  ports:
    - port: 80
      targetPort: 4000
```

17.3 Health check backendu

Po uzyskaniu External IP:

```
BACKEND_LB=$(kubectl get svc backend-service -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
curl "http://$BACKEND_LB/api/health"
```

Oczekiwane: {"api":"ok","spanner":"ok"}.

17.4 Frontend Deployment i Service

Deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: room-booking-frontend
  template:
    metadata:
      labels:
        app: room-booking-frontend
    spec:
      containers:
        - name: web
          image: $FRONTEND_IMAGE
          ports:
            - containerPort: 80

```

Service:

```

apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  type: LoadBalancer
  selector:
    app: room-booking-frontend
  ports:
    - port: 80
      targetPort: 80

```

Uzyskanie adresu:

```

FRONTEND_LB=$(kubectl get svc frontend-service -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "http://$FRONTEND_LB"

```

17.5 Uwaga o pobieraniu obrazów (ImagePullBackOff)

Jesli pojawi sie ImagePullBackOff, nadaj roli reader dla node service account:

```

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${PROJECT_ID}-compute@developer.gserviceaccount.com" \
  --role="roles/artifactregistry.reader"

```

18. Obserwowalnosc (Monitoring i Logging)

18.1 Konfiguracja Cloud Monitoring

GKE domyslnie integruje sie z Cloud Operations.

Zalecane:

- Upewnic sie, ze monitoring i logging sa wlaczone dla klastra.
- Zdefiniowac alerty dla bledow polaczenia z Cloud SQL i Spanner.
- Monitorowac metryki: CPU, pamiec, liczba zapytan DB, opoznienia.

18.2 Budzety i alerty kosztowe

W konsoli Cloud Billing ustaw budzety i alerty procentowe (np. 50%, 90%, 100%).

Pozwala to kontrolowac koszty GKE, Cloud SQL i Spanner.

19. Walidacja i checklisty

19.1 Weryfikacja zasobow

```

gcloud sql instances list

gcloud spanner instances list

gcloud container clusters list

```

```
gcloud artifacts repositories list
```

19.2 Kontrola dostępu

- Czy GSA ma role `cloudsql.client` i `spanner.databaseUser`?
- Czy KSA jest poprawnie zmapowane do GSA?

19.3 Polaczenia testowe

Na etapie wdrozenia aplikacji mozna wykonac test:

- backend -> Cloud SQL (zapytanie `SELECT 1`)
- backend -> Spanner (zapytanie `SELECT 1`)

19.4 Limity i quota

Przed wdrozeniem sprawdz limity w regionie (CPU, liczba adresow IP, liczba instancji):

```
gcloud compute regions describe $env:REGION
```

20. Aneks: zestaw polecen z parametrami

20.1 Zmienne globalne

```
$env:PROJECT_ID="room-booking-prod"
$env:REGION="europe-central2"
$env:SPANNER_CONFIG="regional-europe-central2"
```

20.2 Wlaczanie API

```
gcloud services enable container.googleapis.com sqladmin.googleapis.com spanner.googleapis.com
artifactregistry.googleapis.com iam.googleapis.com
```

20.3 Cloud SQL

```
gcloud sql instances create room-booking-sql --database-version=POSTGRES_15 --cpu=1
--memory=3840MiB --region=$env:REGION
```

```
gcloud sql databases create room_booking --instance=room-booking-sql
```

```
gcloud sql users create room_user --instance=room-booking-sql --password=STRONG_PASS
```

20.4 Spanner

```
gcloud spanner instances create room-booking-global --config=$env:SPANNER_CONFIG --nodes=1
--description="Room Booking Global"
```

```
gcloud spanner databases create room_booking_global --instance=room-booking-global
--ddl-file=spanner.ddl
```

20.5 GKE

```
gcloud container clusters create room-booking-gke --region=$env:REGION --num-nodes=2
--workload-pool=$env:PROJECT_ID.svc.id.goog --release-channel=regular
```

```
kubectl create serviceaccount backend-sa
```

```
kubectl annotate serviceaccount backend-sa
iam.gke.io/gcp-service-account=room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com
```

```
gcloud iam service-accounts add-iam-policy-binding
room-booking-backend@$env:PROJECT_ID.iam.gserviceaccount.com
--role=roles/iam.workloadIdentityUser
--member="serviceAccount:$env:PROJECT_ID.svc.id.goog[default/backend-sa]"
```

20.6 Cloud Build i wdrozenie

```
gcloud services enable cloudbuild.googleapis.com
```

```
gcloud builds submit --config cloudbuild-backend.yaml .
```

```
gcloud builds submit --config cloudbuild-frontend.yaml .
```

```
kubectl apply -f k8s/backend.yaml
```

```
kubectl apply -f k8s/frontend.yaml
```

Koniec dokumentu.